

2PYBOT LAB

Balancing-Robot Simulation Workbench — Lab Manual

Physics · LQR & PID control · fault injection · experiments · every setting explained

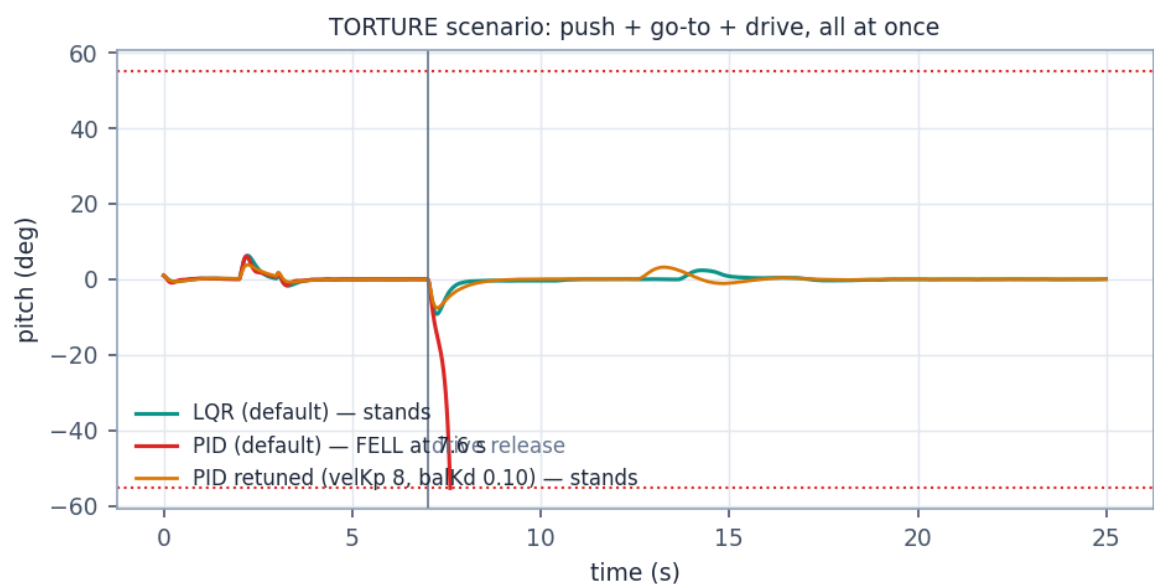


Figure 1. The experiment this tool exists for: identical torture test, three controllers. LQR stands untouched, default PID falls at the drive release, retuned PID recovers — the whole story of the project in one plot.

Applies to	2pybot-lab.html v3 (single-file offline app, 3D viewport + video export)
Modeled robot	2PyBot: ESP32 BaseLink · NEMA17 + TMC2208 steppers · MT6816 encoders 4096 CPR · ISM6HG IMU · 70 mm wheels
Engine validation	Riccati solver matches SciPy to 4 decimals; fault reproductions match the real robot's failures
Manual version	1.0 — August 2026

Contents

- 1 Introduction — what this lab is
- 2 Quick start (five minutes)
- 3 The machine and its physics
- 4 The simulation pipeline — what is actually modeled
- 5 The two controllers: LQR and PID
- 6 Tour of the interface
- 7 How to read the graphs (worked example)
- 8 Configuration reference — every parameter
- 9 Guided experiments E1-E10
- 10 Parameter sweeps — finding the cliff
- 11 Data export, sim video, and loading your CAD
- 12 Under the hood: fidelity, validation, limitations
- 13 Troubleshooting and FAQ
- 14 Glossary

1 Introduction — what this lab is

2PyBot Lab is a self-contained simulation workbench for a two-wheeled balancing robot. It runs the same control laws as the real 2PyBot firmware (BaseLink_LQR and BaseLink_PID), on a physics model that includes the unglamorous parts that actually decide whether a robot stands: sensor noise, filters, encoder quantization, actuator speed and acceleration limits, motor stall, control latency, and sign conventions. Every one of those imperfections is a knob you can turn.

The Lab exists because of a hard-won lesson: this robot's real failures were never exotic. They were an inverted gyro sign, a microstepping mismatch, and a motor stall — and each of those is reproducible here, quantitatively. The simulated inverted-gyro robot whips over in 0.285 s; the real one did it in about 0.3 s. When a tool predicts your bugs that precisely, you can trust it to predict your fixes.

What you can do with it: design LQR gains for any geometry by solving the Riccati equation in the browser, tune the PID cascade by hand, replay every run as a live 3D animation, record simulation videos, compare runs against a pinned baseline, inject faults, and sweep any parameter to find exactly where a design breaks. It is one HTML file, works offline, and stores nothing outside your downloads.

2 Quick start (five minutes)

1. Open `2pybot-lab.html` in any modern browser (Chrome or Edge recommended — required for video export). The app immediately runs the default scenario: your robot, station keeping, LQR.
2. Watch the playback: the 3D robot twitches, catches its 1° initial tilt, and stands. The scope traces draw themselves; the verdict stamps STANDS with six metrics.
3. In the left rail open Fault injection, tick Invert gyro rate (bug), press RUN (or key R). Watch the robot whip itself over in a third of a second — this is the actual bug the real robot shipped with.
4. Untick the fault. Open Scenario, choose Push recovery, RUN. Drag the viewport to orbit the 3D camera while it recovers; scrub the timeline slider to inspect the exact moment the push lands.
5. Press Pin as baseline, switch the top-bar toggle to PID, RUN again. The dashed overlay is LQR; the amber trace is PID; the tiles show the deltas. You have now used every core workflow in the tool.

3 The machine and its physics

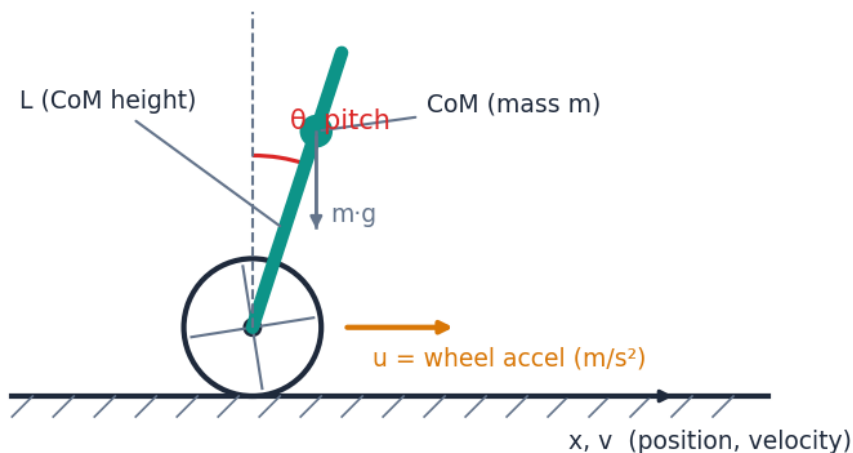


Figure 2. The model: an inverted pendulum whose pivot (the wheel axle) you can accelerate horizontally. Gravity torques the body away from upright; the only tool the controller has is wheel acceleration u .

The robot is an inverted pendulum on wheels. Its body pivots freely about the wheel axle; gravity constantly tries to tip it. The controller cannot push the body directly — it can only accelerate the wheels along the ground. Accelerating the wheels forward makes the body pitch backward relative to them (the same reason you lurch when a bus pulls away), and that coupling is the entire mechanism of balance: drive the wheels back under the falling body, always, 200 times per second.

States

Symbol	Name	Unit	Measured by
x	wheel position along the ground	m	encoders (4096 counts/rev, quantized)
v	wheel velocity	m/s	differenced encoder counts, low-pass filtered at 15 Hz
θ	body pitch from upright (+ = leaning forward)	rad (shown in deg)	IMU + Mahony fusion, two-stage 50 Hz filter
ω	pitch rate	rad/s (deg/s)	gyro, direct
z	integral of position error over time	m·s	computed by the controller

Equations of motion (as simulated)

$\omega' = (g \cdot \sin(\theta) - a \cdot \cos(\theta)) / L$ body angular acceleration
 $x' = v, \quad v' = a$ wheel kinematics (stepper-driven)
 a = wheel acceleration actually delivered by the steppers

L is the height of the center of mass above the axle — the single most important number in the model. It sets the natural toppling rate $\sqrt{g/L} \approx 8.1$ rad/s at $L = 0.15$ m: left alone, a small tilt grows e-fold every 0.12 s. Everything the controller does must be faster than that.

Why this plant is genuinely hard: non-minimum phase

To move forward from rest, the robot must first lean forward; to lean forward, the wheels must first roll backward. So the correct response to “go right” begins with a small move left. Systems like this are called non-minimum phase, and they are why intuitive hand-tuning fights you: the mathematically correct gains command initial motion that looks wrong. You can see the effect

directly in Figure 8 (drive experiment): the wheels dip backward before every acceleration. It also explains why all five LQR gains come out negative — that is the physics, not a sign error.

4 The simulation pipeline — what is actually modeled

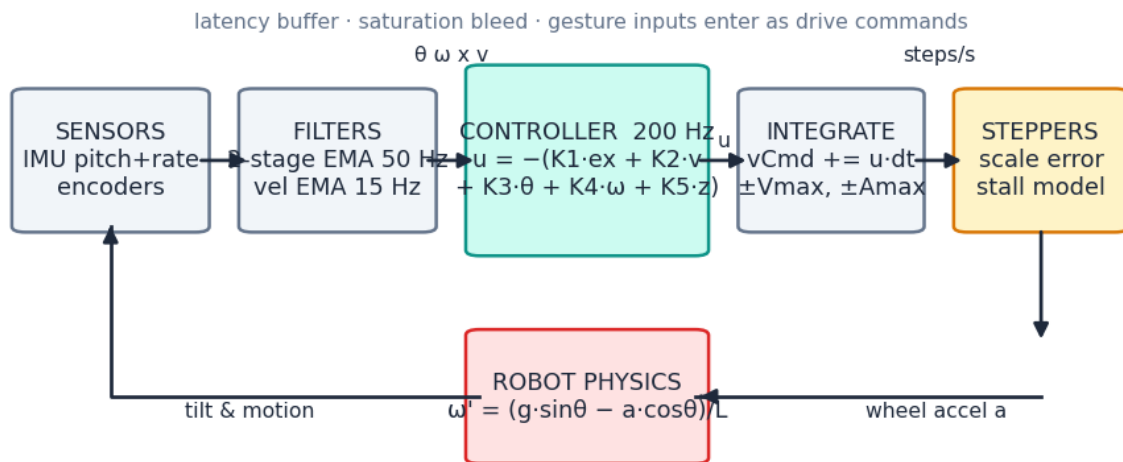


Figure 3. One control period (5 ms at 200 Hz). Physics integrates in 10 substeps between controller ticks. Every block in this chain is configurable, and every block has broken at least one real robot.

Stage	What the Lab models	Where configured
Pitch sensing	Gaussian angle noise + a constant CoG offset (the robot's true balance point is not the IMU's zero), then a 1- or 2-stage exponential filter at a settable cutoff. Filters add phase lag — stability costs.	Sensors & timing
Rate sensing	Gyro noise + constant bias. Sign can be inverted as a fault.	Sensors, Faults
Odometry	Position quantized to encoder counts; velocity from differenced counts through a low-pass. Sample dropout injectable. Sign invertible as a fault.	Sensors, Faults
Controller	LQR full-state feedback with integral action, or the PID cascade — the exact firmware structures, including position-error clamp, integral clamps, and the saturation ground-giving safeguard.	LQR / PID panel
Latency	The control output can be delayed N ms (buffer), emulating slow loops or communication lag.	Motor & drive
Actuator	u integrates into a velocity command, clamped to the speed cap; the wheel slews toward it under the acceleration limit. A scale error multiplies real motion per commanded step (microstep mismatch). Above the stall speed the motor loses sync: torque vanishes and the wheel spins down until the command returns below recovery.	Motor & drive
Disturbances	Rate impulses (pushes), continuous random gusts, initial tilt, and the scenario timeline.	Scenario

Determinism: the noise generator is seeded (Simulation → Noise seed), so a run is exactly repeatable — change one parameter and any difference you see is caused by that parameter, not by luck. Change the seed to check that a marginal design is not just a lucky roll.

5 The two controllers: LQR and PID

5.1 LQR — Linear Quadratic Regulator (with integral action)

```

u = -( K1·ex + K2·v + K3·θ + K4·ω + K5·z )
ex = clamp(x - x_target, ±0.15 m)      z = ∫ ex dt (clamped)

```

One line, five feedback terms, each watching one state. The magic is where the K values come from: you do not choose them. You choose weights — how much you dislike each kind of error (Q1...Q5) and how much you dislike motor effort (R) — and the solver computes the gain vector that minimizes the total lifetime cost. That computation (the algebraic Riccati equation, solved in the app by Newton iteration seeded with pole placement) produces gains that are consistent with each other by construction, which is precisely what hand-tuning five coupled knobs cannot guarantee.

Gain	Watches	Acts like	Default (this robot)
K1	distance from target	spring to the hold spot	-12.74
K2	wheel velocity	damping on travel	-10.70
K3	pitch angle	the main upright spring	-48.28
K4	pitch rate	shock absorber on tilt	-6.01
K5	accumulated position error	slow, patient drift-killer (this is the “I” in LQI)	-4.47

Workflow: set your robot's L in Robot model, set Q and R in the LQR panel, press Solve Riccati → K, and watch the Newton iterations converge (13 steps for the default). The closed-loop poles then appear as chips: every pole must be negative (green). Poles are the decay rates of the balanced system — the default set is -17.6 , -7.8 , $-1.73 \pm 0.87j$, -0.57 , meaning a fast tilt-catch layer, a medium recovery layer, and a slow, gentle position layer. A pole at -2 halves its error every 0.35 s; more negative = faster but hungrier for actuator. All K fields stay editable if you want to hand-perturb the optimum and watch what breaks.

5.2 PID — the classical cascade

```

vRef      = clamp( posKp · (x_target - x), ±vHoldMax )      position loop
pitchRef  = clamp( velKp·(vRef - v) + velKi·∫(vRef - v)dt ) velocity loop
u         = balKp·(θ - pitchRef) + balKd·ω                  angle loop

```

Three nested loops: position asks for a velocity, velocity asks for a lean, lean asks for wheel acceleration. Each loop must be substantially slower than the one inside it or they fight. The Lab's defaults stand comfortably in every single scenario except Torture — deliberately kept that way, because re-tuning PID to survive Torture (Experiment E5) teaches more about cascade controllers than any paragraph can.

5.3 Which to use

Use the Lab's baseline-pinning to answer this empirically rather than religiously. Broadly: LQR wins when you can measure your robot (L, mainly) and want coordinated aggression with robustness margins; PID wins when you have no model at all and are willing to iterate. On the real 2PyBot, LQR balanced on roughly the second attempt after a thousand failed PID hand-tunes — but note that the old firmware's PID also had a structurally broken architecture, so the fair fight is the one you can run here.

6 Tour of the interface

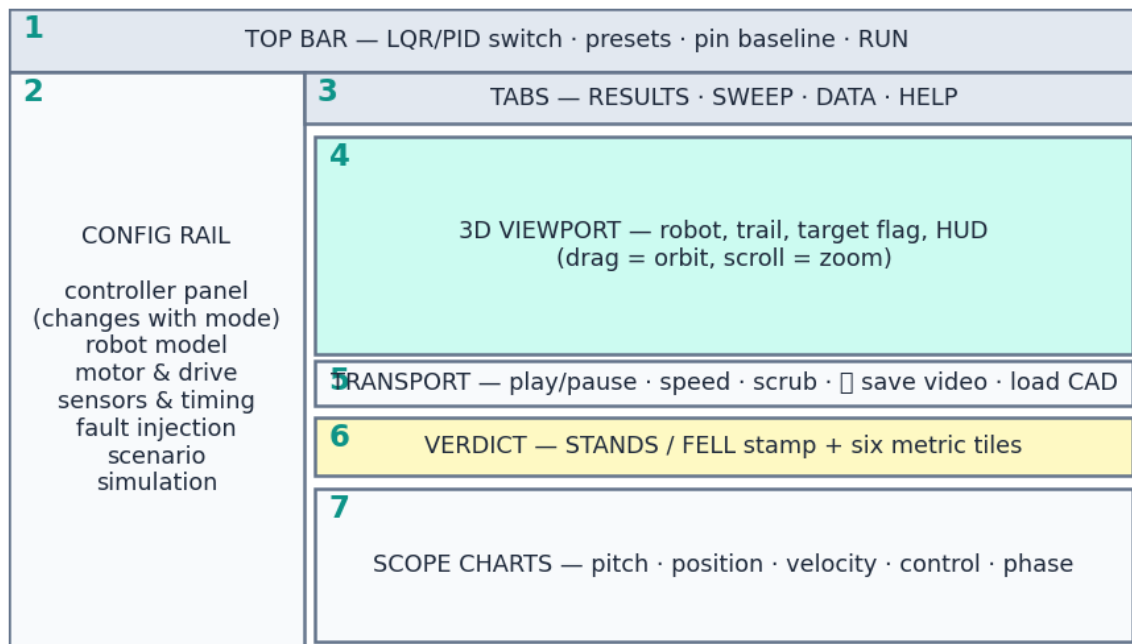


Figure 4. The seven regions of the app. The accent color follows the controller: teal = LQR, amber = PID.

#	Region	What it does
1	Top bar	Controller toggle (rebuilds the rail), preset picker (including reproductions of every real bug), Pin/Clear baseline, and RUN.
2	Config rail	Every parameter, grouped in collapsible sections. The controller panel at the top changes with the mode and hosts the Riccati solver (LQR) or the cascade gains (PID). Hover any dotted-underlined label for a tooltip.
3	Tabs	RESULTS (viewport + verdict + charts), SWEEP, DATA (exports), HELP (built-in reference + self-test).
4	3D viewport	The robot itself: spoked wheels that roll, tilting body, CoM marker, target pennant, trajectory trail, HUD readout, red SAT lamp when the speed command saturates. Drag = orbit, scroll = zoom. Falls play out — the body tips onto the grid with a FELL overlay.
5	Transport	Play/pause (spacebar), speed 0.25×–8× or instant, scrub slider, Save video, Load CAD (.stl).
6	Verdict	The STANDS/FELL stamp and six metric tiles (defined in §7.3). With a pinned baseline, tiles show deltas: green = improved, red = worse.
7	Scope charts	Pitch, position vs target, velocity commanded-vs-actual, control effort, and the phase portrait. During playback they draw progressively with a time cursor; axes are pre-locked to the full run so traces grow into stable frames.

Presets

Preset	What it demonstrates
2PyBot — your robot	The real machine: L 0.15 m, 70 mm wheels, 1/8 microstepping, 6000 steps/s cap, stall 9 k, 4096 CPR.
Ideal robot	No noise, no CoG bias, no stall, huge limits — what control theory papers quietly assume.
Tall robot (L = 0.30)	Same gains, doubled CoM height: slower toppling but more inertia. Re-solve K and compare.
Bug: inverted gyro rate	The real 18-Aug failure. Anti-damping; falls in 0.285 s.

Preset	What it demonstrates
Bug: inverted pitch sign	The classic sign error: wheels drive into the fall.
Bug: stall at 9k, cap 18k	The real motor-stall failure under a push.
Bug: 2× scale	The real microstep mismatch (drivers at 1/8, firmware assuming 1/16).
Sloppy sensors + latency	0.5° angle noise, 2°/s gyro noise, 15 ms delay: how much imperfection the design tolerates.

Keyboard

Key	Action
R	run the current configuration
Space	play / pause the playback
drag / scroll on viewport	orbit / zoom the 3D camera

7 How to read the graphs (worked example)

This section does exactly one thing: takes a real output graph and explains every mark on it, then defines every term the app reports. The example is the PITCH chart from a push-recovery run — the single most information-dense plot the tool produces.

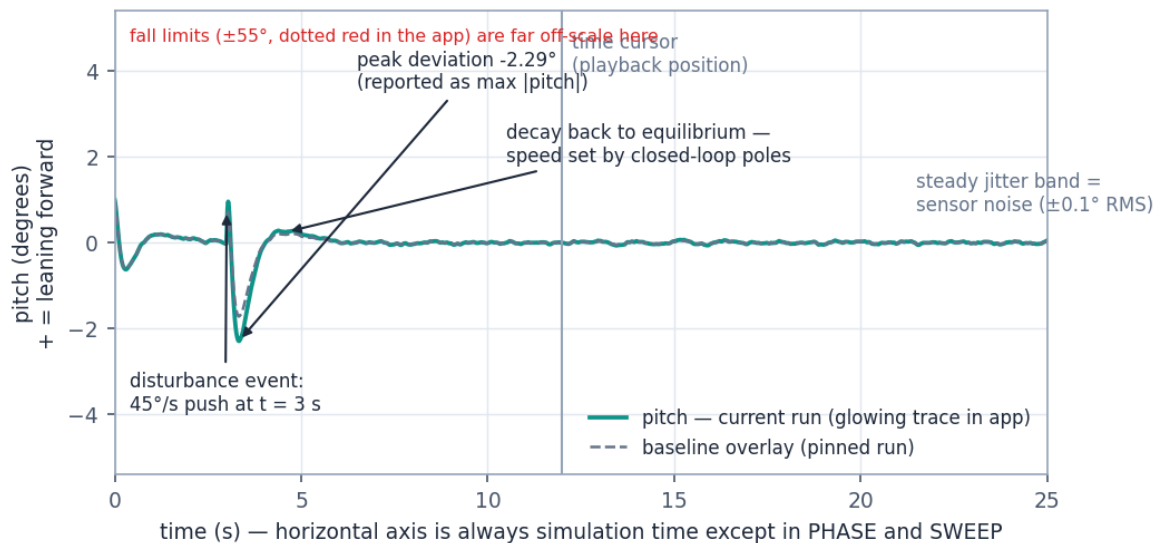


Figure 5. Anatomy of a pitch chart (push recovery, 45°/s at $t = 3$ s). Every annotated element appears in the live app: the glowing teal trace is the current run, the dashed gray is the pinned baseline, the vertical line is the playback cursor, and the dotted red fall limits sit at $\pm 55^\circ$, far outside this healthy run's window.

7.1 Reading it, mark by mark

Horizontal axis — time (s). Simulation time, not wall time; the playback speed only changes how fast you watch it. Vertical axis — pitch (degrees), + = leaning forward. Zero is upright. The initial transient ($t < 0.5$ s) is the robot catching its 1° release tilt — a brief dip as the wheels first roll the wrong way (non-minimum phase, §3) and then a clean return. The disturbance at $t = 3$ s is a step in pitch rate, not angle — like a sharp flick: the trace leaves zero at high slope, peaks at -2.29° , and decays. The peak deviation is what the verdict reports as $\max |pitch|$: the single worst moment of the run. The decay back to zero takes about 2 s and its shape is set by the closed-loop poles: the fast poles kill the rate, the $-1.73 \pm 0.87j$ pair gives the one small overshoot wiggle, the slow pole drifts the last fraction home. The steady band after recovery is $\pm 0.2^\circ$ -ish jitter — that is your sensor noise being faithfully passed through the controller, and it is the noise floor every metric sits on.

7.2 The other charts, briefly

Chart	What to look for
POSITION (m)	Robot (glowing) vs target (gray). Healthy hold: flat lines a hair apart. A constant gap = weak integral. A slow saw-tooth = integral pumping (K5 too strong).
VELOCITY (m/s)	Commanded (gray) vs actual (accent) with the $\pm V_{\max}$ cap lines. The two traces should hug; actual collapsing toward zero while commanded rails is the motor-stall signature (Fig. 12). Time pinned at the cap = danger even if the run stood — see the SAT metric.
CONTROL u (m/s ²)	The controller's effort with $\pm A_{\max}$ limits. Grass-like noise around zero is normal (it is amplified sensor noise). Flat-topping at the limits means the controller wants more actuator than it has.
PHASE (θ vs ω)	Time is hidden; the trajectory of (tilt, tilt-rate) is shown. Healthy runs spiral into the origin (Fig. 13). Loops that grow outward = instability; closed loops that never shrink = a limit cycle (something oscillating forever); a straight shot off the page = a sign error.
SWEEP	One point per full simulation; the x-axis is the swept parameter, not time. Red dots = runs that fell.

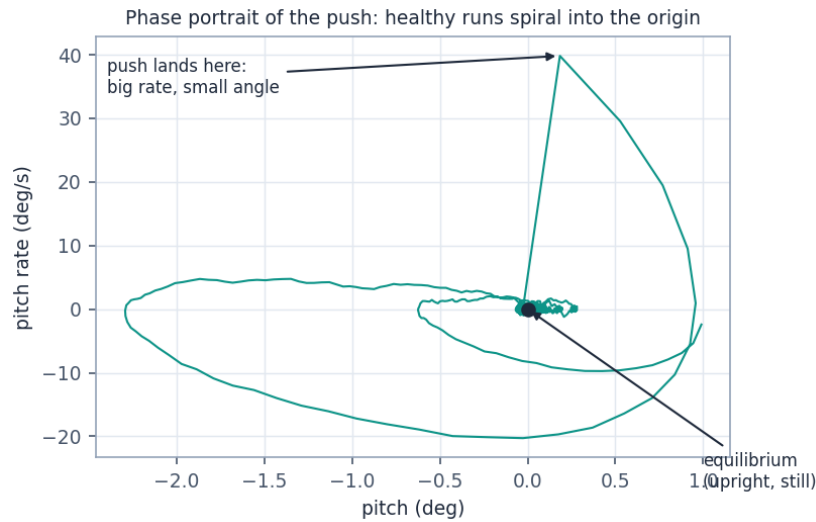


Figure 6. The push in phase space. The impulse teleports the state vertically (rate changes instantly, angle hasn't yet); the controller then spirals it back to the origin. Damping = how directly it heads home.

7.3 The verdict metrics, defined

Title	Definition	Healthy (this robot, station)
STANDS / FELL	Fell = $ \text{pitch} $ exceeded the fall cutoff (default 55°). Time of death is shown.	STANDS
max $ \text{pitch} $	Largest absolute pitch anywhere in the run — the worst moment.	$\approx 1^\circ$ release / $\approx 2.3^\circ$ for a $45^\circ/\text{s}$ push
RMS position	Root-mean-square distance from target over the second half of the run (transients excluded). The honest 'how well does it hold station' number: $\text{RMS} = \sqrt{\text{mean of error}^2}$, so occasional large errors are punished more than the plain average would.	< 0.1 cm
max excursion	Largest position error anywhere, transients included.	a few cm
settled < 1 cm	First time after which position error never again exceeds 1 cm. '—' if it never settles.	seconds
speed-rail time (SAT)	Percent of the run with the velocity command above 90% of the cap: how close to actuator exhaustion the run lived. High SAT with STANDS is a warning, not a pass.	0%
top speed	Peak $ \text{velocity command} $ — how much actuator the run actually used.	well under V_{max} 0.82 m/s

8 Configuration reference — every parameter

Grouped exactly as in the rail. “Too low / too high” describes the symptom you will see in the charts.

Robot model

Parameter	Unit	Default	Meaning · symptoms
CoM height L	m	0.15	Height of center of mass above the axle. Gains are designed for it — change it, re-solve K. Higher L = slower toppling but lazier robot; running old gains on a very different L degrades damping (sweep it: Fig. 14).
CoG offset	deg	0.35	How far the true balance angle sits from the IMU's zero. The integral (K5) absorbs it; the robot stands with a permanent tiny lean. Large values eat actuator headroom and slow settling.
Wheel radius	m	0.035	Converts wheel revolutions to meters. Wrong radius = wrong loop scale: everything the controller believes about speed and distance is off by the same factor.
Fall cutoff	deg	55	pitch beyond this = FELL, run ends. Matches the firmware's cutoff.

Motor & drive

Parameter	Unit	Default	Meaning · symptoms
Full steps per rev	—	200	Motor constant (1.8° steppers).
Microstepping	—	8	Divides each step. Together with steps/rev and wheel radius it sets steps-per-meter — the scale of every command.
Speed cap	steps/s	6000	The velocity command clamp (= 0.82 m/s here). Too low: the robot runs out of recovery authority (watch SAT%). Above the stall speed: see stall.
Acceleration limit	steps/s ²	20000	Clamp on u (= 2.75 m/s ²). Too low: soft, mushy catches, falls to modest pushes. Very high: nothing in sim, but real steppers skip.
Stall speed	steps/s	9000	Above this commanded rate the motor loses sync: torque gone, wheel spins down, recovers only below the recovery fraction. 0 disables. This killed the real robot — Experiment E8.
Stall recovery	fraction	0.75	Command must fall below recovery×stall for the motor to re-engage.
Actuation scale error	×	1.0	Real motion per commanded step vs firmware belief. 2.0 = the real 1/8-vs-1/16 microstep bug: the whole loop runs at double gain. Watch velocity: actual = 2× commanded.
Control latency	ms	0	Delay between computing u and it acting. Sweep it (E10): stability dies somewhere in the tens of ms — a vivid demonstration of why the loop runs at 200 Hz.

Sensors & timing

Parameter	Unit	Default	Meaning · symptoms
Control loop rate	Hz	200	Controller tick rate. Halving it roughly halves the tolerable latency and doubles discretization phase lag.
IMU angle noise	deg RMS	0.10	Gaussian noise on measured pitch. Shows up directly as the jitter band and as grass on u.
Gyro noise / bias	deg/s	0.30 / 0	Noise passes through K4 into u; constant bias tilts the equilibrium slightly (integral cleans up position).

Parameter	Unit	Default	Meaning · symptoms
Pitch filter cutoff / stages	Hz	50 / 2	The firmware's smoothing. Lower cutoff = calmer signal but more phase lag — at some point lag destabilizes faster than smoothing helps. Try 10 Hz.
Encoder resolution	counts/rev	4096	Position quantization. Very coarse encoders make velocity estimates steppy — watch the velocity trace get chunky at 256.
Velocity filter cutoff	Hz	15	Low-pass on differenced counts. Trade smoothness against lag in the v estimate.

Fault injection

Fault	What it breaks	Signature
Invert pitch sign	Wheels drive into the fall — full positive feedback on angle.	Whip-over in $\approx \frac{1}{4}$ s from any tilt.
Invert gyro rate	K4 damping becomes anti-damping; angle term still correct.	Violent growing oscillation; over in 0.285 s. The real robot's actual bug.
Invert encoder sign	Position/velocity feedback backwards; the fast tilt loop is fine.	Balances briefly, then lunges fore-aft with growing amplitude; falls in ≈ 2.6 s. Slow failure = outer loop.
Encoder dropout %	Random held samples in odometry.	Velocity spikes; at high % the position loop wanders.

Scenario & simulation

Parameter	Meaning
Test	station (hold a spot) · push (rate impulse at a set time/size) · step (go 0.5 m out and back) · drive (velocity command window, then release) · torture (all of the above simultaneously).
Duration / initial tilt	Run length; the tilt the robot is 'released' at (arming error).
Push time / size	When the impulse hits and how hard, in deg/s of instantly added pitch rate.
Go-to distance / times	Step target and its out/return schedule.
Drive speed / window	Commanded velocity and its start/end.
Random gusts	Continuous white torque disturbance (deg/s ² RMS) — simulated wind.
Full-stick speed / deadband	Drive-command scaling, mirroring the firmware's joystick mapping.
Physics substeps	Integration substeps per control tick (10 = 2 kHz physics). Raise if you push parameters to extremes.
Noise seed	Repeatability. Same seed = identical noise; change it to test luck-sensitivity.

LQR panel / PID panel

Parameter	Meaning
Q1...Q5 (LQR)	Cost weights: position, velocity, tilt, tilt-rate, integral. Relative size is what matters — double Q3 and the solver stiffens the tilt response at the expense of everything else.
R (LQR)	Cost of actuator effort. Smaller R = more aggressive gains everywhere. R is the single best 'overall aggression' knob.

Parameter	Meaning
Position error clamp / integral clamp	The firmware's balance-first safeguards: a far-away target asks for a walk, not a sprint.
Give ground when saturated	The anti-fall bleed: at the speed rail, the hold point is dragged to the robot. Turn it off and re-run a hard push against a low speed cap to watch why it exists.
Brake lookahead	Where the hold point latches when a drive command ends (predicts the stopping point).
posKp / vHoldMax (PID)	Position loop gain and its velocity request cap.
velKp / velKi / clamp (PID)	Velocity loop: proportional lean per m/s of error, integral lean per meter of accumulated error.
balKp / balKd / lean limit (PID)	The innermost angle loop and the maximum lean the outer loops may request.

9 Guided experiments E1-E10

Each experiment states the setup, what you should observe, and why. All figures below are actual Lab output at defaults.

E1 — Station keeping (the baseline)

Setup: preset '2PyBot', scenario Station, RUN. Observe: the release transient, then a flat hold: RMS position 0.05 cm, max pitch $\approx 1^\circ$ (the release itself). Why it matters: this is your control group — pin it as baseline before every other experiment.

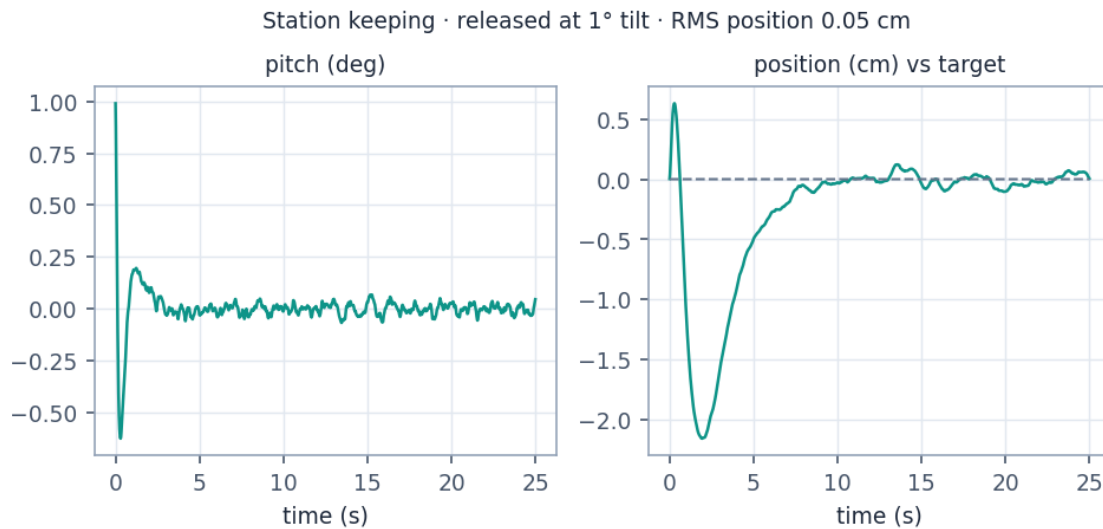


Figure 7. E1: the robot catches a 1° release and parks. The position dip at the start is the non-minimum-phase back-roll; the integral then walks it home.

E2 — Push recovery

Setup: scenario Push ($45^\circ/\text{s}$ at $t = 3$ s). Observe: peak $\approx 2.3^\circ$, recovered inside 2 s, u briefly slams toward its limit then relaxes. Try: raise the push until it falls (the cliff is near $120^\circ/\text{s}$ — Fig. 14 right); watch how SAT% climbs first. Falling only after saturating is the signature of an actuator-limited (not controller-limited) design.

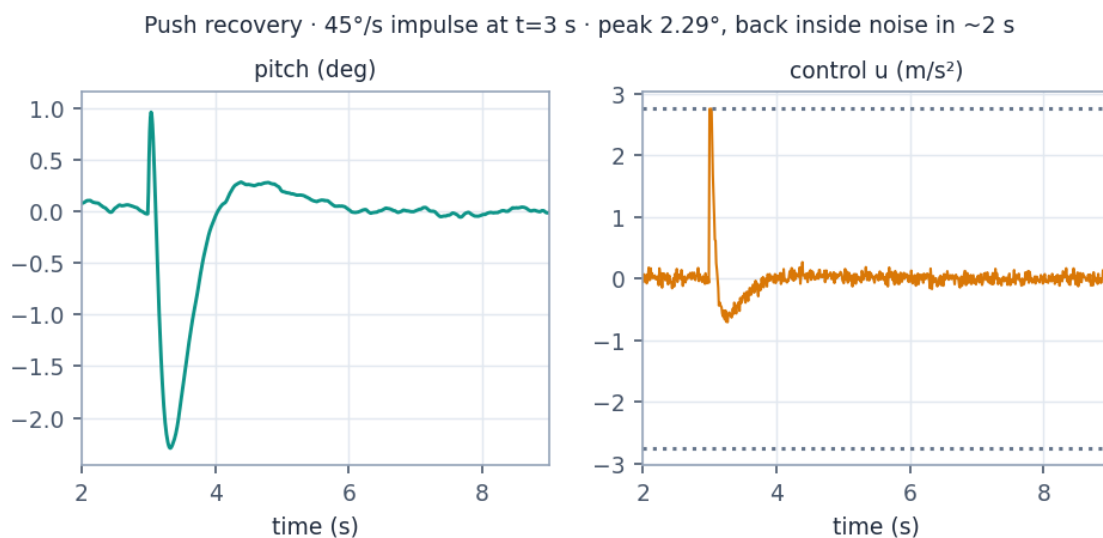


Figure 8. E2: pitch and control effort during the push. Note u touching its clamp — recovery authority is finite, and the SAT lamp in the viewport marks every moment at the rail.

E3 — Position go-to

Setup: scenario Position go-to. Observe: the robot leans into each move and brakes with a counter-lean — the pitch trace is the mirror of the acceleration profile. Steady-state error after arrival $\leq$ noise, courtesy of K5.

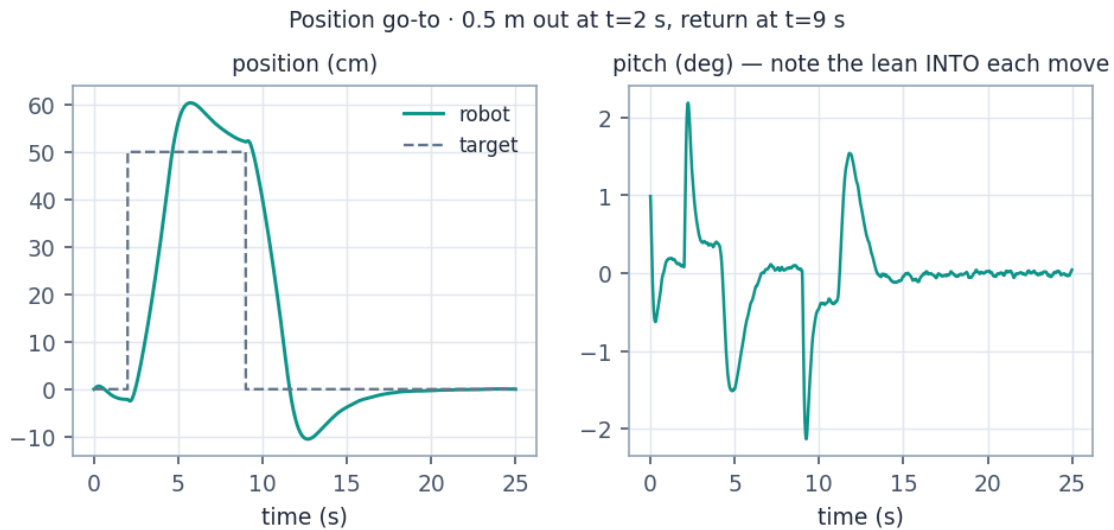


Figure 9. E3: 0.5 m out and back. Balancing robots translate by falling on purpose.

E4 — Drive & release, and seeing non-minimum phase

Setup: scenario Drive & release. Observe (Fig. 8 right): at $t = 2$ s the wheels first roll backward ~ 4 cm to pitch the body into the drive, then accelerate. At release the hold latches near the stopping point.

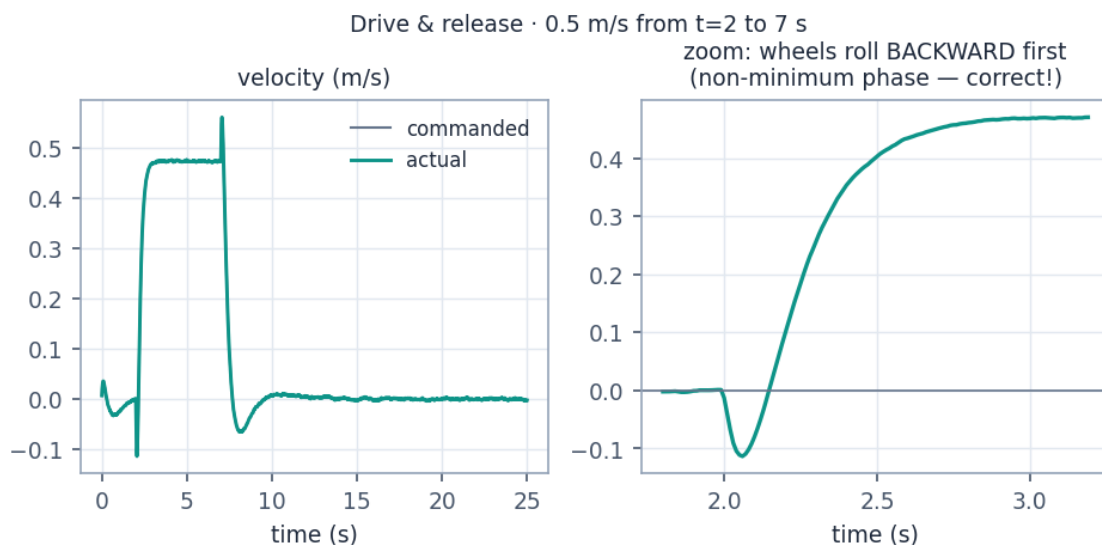


Figure 10. E4: commanded vs actual velocity, and the zoomed backward dip that proves the plant is non-minimum phase. If your intuition says this is a bug — that is exactly why hand-tuning is hard.

E5 — LQR vs PID, honestly (the Torture challenge)

Setup: scenario Torture, LQR, RUN, Pin as baseline. Switch to PID, RUN. Observe: default PID dies at the drive release ($t = 7.6$ s): the cascade demands a violent velocity reversal, over-leans, and never recovers. Now fix it: Velocity P 10 $\rightarrow$ 8, Angle D 0.070 $\rightarrow$ 0.10, RUN — it stands (Fig. on cover / Fig. 9). Lesson: PID could always do it; a human just has to re-coordinate the loops for each new situation. LQR's solver does that coordination once, globally.

E6 — Design your own gains (Q/R lab)

Setup: LQR panel. Try three designs, solving and running Push after each: (a) Soft: $R = 1.0$ — gentle gains, larger peak, calm u . (b) Stiff: $Q3 = 300$ — tiny peaks, grassy u , buzzy in the viewport. (c) Position-obsessed: $Q1 = 200$, $Q5 = 60$ — this is literally the firmware's STIFF-HOLD gain set. Watch the poles move with every solve: your weights are sculpting decay rates. Rule of thumb: weights are relative; only ratios matter.

E7 — Fault lab: the inverted gyro (the real bug)

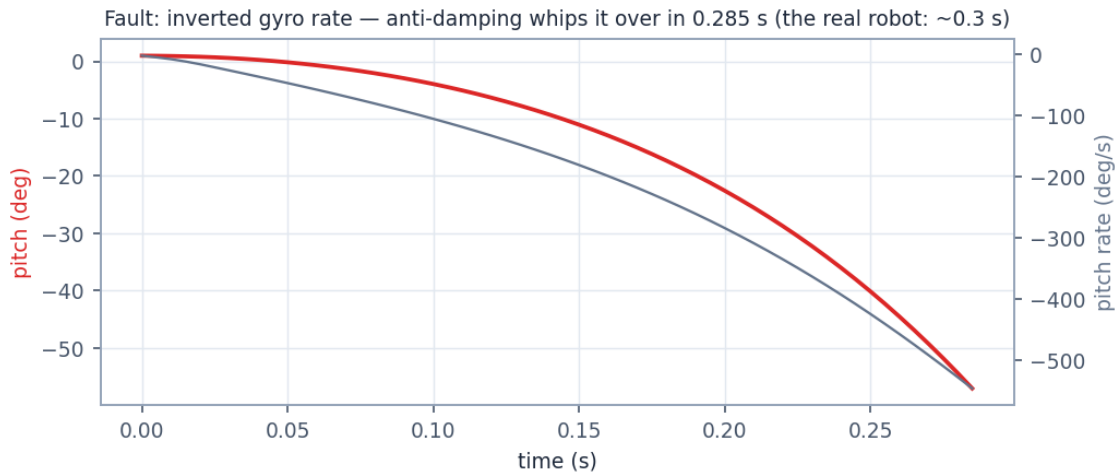


Figure 11. E7: with the rate sign flipped, every swing is amplified instead of damped — energy pumping. The simulated fall at 0.285 s matched the real robot's video to within a frame or two, which is how the bug was found from a serial log.

Setup: Faults → Invert gyro rate. Observe: pitch and rate move in opposite phase (the annotated giveaway: $w = -d\theta/dt$), oscillation grows, over in a third of a second. Also try: Invert pitch sign (falls even faster, no oscillation — pure positive feedback) and Invert encoder (slow fore-aft lunge, ~2.6 s — an outer-loop failure). Diagnostic rule this teaches: fast failures live in the tilt path, slow failures live in the position path.

E8 — Fault lab: motor stall

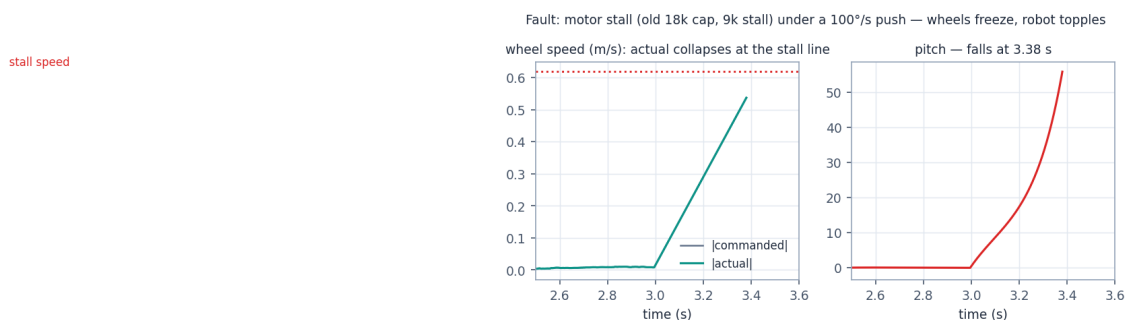


Figure 12. E8: the actual wheel speed tracks the command until it crosses the stall line, then collapses to zero while the command rails — a balancing robot whose wheels freeze topples immediately.

Setup: preset 'Bug: stall at 9k, cap 18k' (raise the push to ~ $100^\circ/s$). Observe: the velocity chart's scissoring traces — the exact signature that appeared in the real robot's logs, where v collapsed to 0.02 m/s while v_{Cmd} railed at 18000 steps/s. Fix it live: set the Speed cap to 6000 (below stall) and re-run: the robot gives up some ground (watch the target pennant walk toward it — that is the saturation bleed) but stands.

E9 — Scale-error lab (the microstep mismatch)

Setup: Actuation scale error = 2.0. Observe: the robot stands — LQR's robustness margins absorb a doubled loop gain — but the velocity chart shows actual = $2\times$ commanded, and the ride is visibly harsher. Push the error to $4\times$ and it buzzes; to $1/16$ (a driver left at 256 microsteps) and it is too

weak to catch anything. Lesson: robustness is a budget; configuration errors spend it before disturbances get any.

E10 — Robustness sweeps

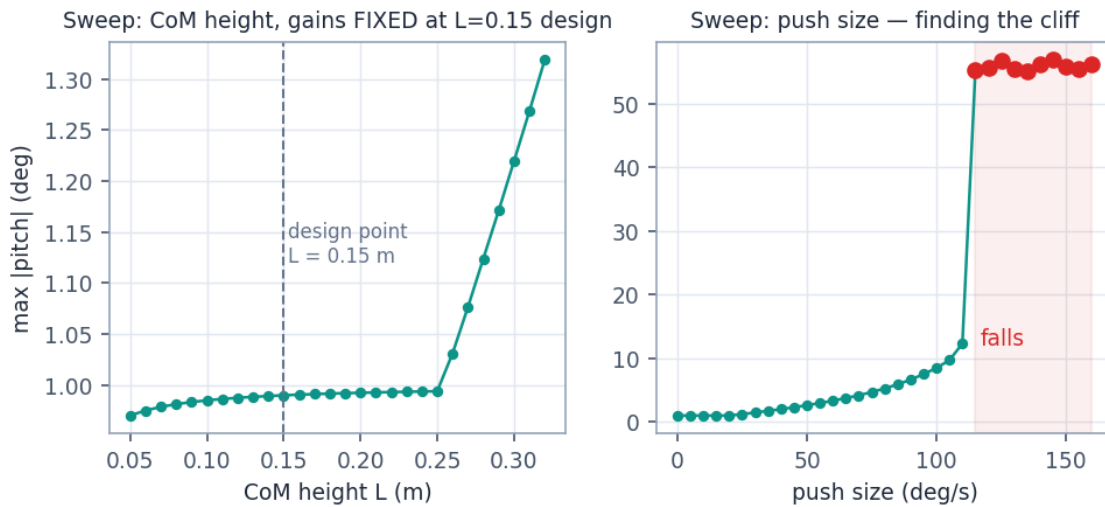


Figure 13. E10: left — the design tolerates a huge CoM-height error before degrading (gains fixed at the $L = 0.15$ design); right — the push cliff: graceful until $\sim 115^\circ/\text{s}$, then dead. Red = fell.

Setup: Sweep tab. Sweep CoM height 0.05→0.32 m (metric $\max|\text{pitch}|$), then push size 0→160. Observe: the flat valley = your robustness margin; the cliff edge = your spec sheet. Sweep Control latency 0→40 ms next and find the stability boundary of the 200 Hz loop — then double the loop rate and watch the boundary move.

10 Parameter sweeps — finding the cliff

A sweep re-runs the current scenario N times, stepping one parameter across a range, and plots one metric per run. Points land live, one simulation at a time, with failed runs marked red and narrated in the status line. Sweepable parameters: CoM height, CoG bias, push size, actuation scale, stall speed, speed cap, loop rate, latency, IMU noise, and an all-K scale factor (multiplies the whole gain vector — the classical gain-margin experiment). Metrics: max |pitch|, RMS position, max excursion, or SAT%. Practical reading: report the flat region as your tolerance, the knee as your operating margin, and never ship a design whose defaults sit near a cliff edge.

11 Data export, sim video, and loading your CAD

CSV traces

Column	Meaning
t	time, s (one row per control tick)
pitch_deg / pitch_rate_degs	body angle and rate
x_m / xRef_m	position and its target (watch xRef move during drives and bleeds)
v_ms / vCmd_ms	actual and commanded velocity — their divergence is the stall/scale diagnostic
u_ms2	control output (wheel acceleration)
saturated	1 while the speed command is above 90% of the cap

DATA tab also saves every chart as PNG (including the 3D viewport) and the entire configuration as JSON — the JSON is the reproducibility artifact: attach it to a bug report or load it back later and the run is bit-identical (same seed).

Sim video

□ Save video restarts the playback from $t = 0$ while recording the 3D viewport, then downloads a .webm when the run ends (fall post-roll included). The recording follows the transport speed: 1× for a real-time clip, 0.25× for slow-motion fall analysis. Requires a Chromium-based browser; orbit the camera while recording for cinematic shots.

Loading your CAD

Load CAD accepts your robot's STL (binary or ASCII). It is assumed Z-up (normal CAD convention), auto-centered, scaled so its height matches the configured CoM geometry, and mounted at the axle pivot so it tilts correctly. It replaces the body only — the wheels stay procedural so they keep spinning. × restores the built-in green-PCB model.

12 Under the hood: fidelity, validation, limitations

Validation (also runnable anytime: HELP → Run self-test)

Check	Result
Riccati solver vs SciPy solve_continuous_are	K matches to 4 decimal places (−12.7422, −10.6993, −48.2848, −6.0051, −4.4721); poles match; converges in ~11 Newton steps, ~15 ms.
Inverted-gyro fall time	Sim 0.285 s vs real robot ≈ 0.3 s (video). Same whip-over morphology.
Motor stall signature	Sim reproduces the real log: actual velocity collapses while the command rails; robot topples.

Check	Result
Scale error	Sim velocity ratio matches the real 1/8-vs-1/16 encoder evidence (actual $\approx 2\times$ commanded).
Scenario matrix	Both controllers, all five scenarios, deterministic outcomes (PID/Torture intentionally falls at defaults).

Deliberate limitations

The model is planar: one translation axis plus pitch. Yaw dynamics, wheel slip, tire compliance, battery sag, motor torque curves (beyond the hard stall), and structural flex are not modeled. The stall model is a binary lose-sync/recover approximation of a smooth torque-speed rolloff. Gestures, heading hold, and the stiff-hold gain switching live in the firmware, not the Lab (though you can design the stiff gain set here — E6c). These omissions are chosen: everything that decided the real robot's life or death is in; everything else would slow the loop of question → run → answer.

13 Troubleshooting and FAQ

Symptom / question	Answer
The app opens but video export does nothing	Canvas recording needs a Chromium browser (Chrome/Edge). Firefox and Safari play everything else fine.
Solve Riccati says UNSTABLE	Your weights are degenerate (e.g., a zero or negative Q entry with tiny R). Keep all $Q > 0$, $R > 0$. If K still looks wild, your L is unphysical (< 2 cm).
Everything falls, even defaults	Check the Faults group — a leftover tick is the usual culprit. Presets reset it.
Why do all K values start with a minus sign?	Physics (§3): the plant is non-minimum phase and the firmware convention is $u = -(K \cdot \text{state})$. Making them 'positive to look right' is exactly the sign error that Experiment E7 demonstrates.
The robot stands but SAT% is high	You are surviving on luck. Raise the speed cap, soften the position gains, or reduce the disturbance — high rail-time means one more push ends it.
My real robot behaves differently from the sim	First verify scale: motors off, roll it 1 m, check encoder counts vs counts-per-meter. Then compare the sim's velocity chart against a real debug log at the same command — the v-vs-vCmd relationship exposes stall, scale, and sign errors in one look.
Playback is choppy on an old laptop	Use instant speed (charts and verdict are unaffected), or reduce duration. The physics itself computes in tens of milliseconds regardless.
Where is my data stored?	Nowhere. The file has no network access and no storage; the only outputs are the files you explicitly download.

14 Glossary

Term	Meaning
LQR / LQI	Linear Quadratic Regulator: full-state feedback with gains computed by minimizing a quadratic cost. LQI adds an integral state (the K5 term here) to remove steady-state error.
Riccati equation	The matrix equation whose solution yields the optimal LQR gains. Solved in-app by Kleinman-Newton iteration seeded with Ackermann pole placement.
Pole	A decay rate of the closed-loop system (units 1/s). Negative = stable; magnitude = speed; imaginary part = oscillation frequency of that mode.
Non-minimum phase	A system whose correct response starts in the 'wrong' direction — here, rolling backward to go forward.

Term	Meaning
Gain / phase margin	How much extra gain or delay the loop tolerates before instability — measured empirically here via the K-scale and latency sweeps.
Saturation	An actuator at its limit. A saturated controller is temporarily open-loop — the most dangerous state a balancer has.
RMS	Root-mean-square: $\sqrt{\text{average of squares}}$. The standard 'typical magnitude' for signals that wander around zero.
Settling time	When the response enters and stays inside a tolerance band (1 cm here).
Dead reckoning / odometry	Position integrated from wheel encoder counts.
EMA filter	Exponential moving average — the one-line low-pass the firmware uses; cutoff sets smoothing vs lag.
Phase portrait	A plot of state vs its derivative (θ vs ω) with time implicit; stability is visible as spiraling inward.
Stall (stepper)	Loss of synchronization between the step commands and the rotor: torque collapses until the command rate drops. Not gradual — a cliff.

— end of manual · generated from real engine runs · 2PyBot Lab v3 —